Transcript.IO

Projet Bases du Web 2026-2027, ENSIIE. Auteur : Siin0pe. Technologie principale : Next.js.

1. Technologies utilisées

- **Next.js 16** (React 19, TypeScript) : une seule application contient la partie client (composants React exécutés dans le navigateur) et la partie serveur (Server Components et Server Actions exécutés par Node.js).
- **Tailwind CSS 4** pour le style, avec des composants shadcn/ui (bouton, champ, liste déroulante).
- **SQLite** avec **Drizzle ORM** et le pilote libSQL. La base est un simple fichier créé au premier lancement à partir de `db/init.sql` : aucune installation de serveur de base de données ni de Docker n'est nécessaire.
- **Whisper tiny** (bibliothèque `browser-whisper`) : reconnaissance vocale exécutée dans le navigateur en WebAssembly.

Base de données : 2 tables. `tags` (étiquettes, avec un nom français et un nom anglais) et `notes` (titre, contenu, note dictée ou non, dates, clé étrangère `tag_id` vers `tags`). Requêtes : `SELECT` avec jointure `LEFT JOIN` pour afficher la liste, `INSERT`, `UPDATE` et `DELETE` pour créer, modifier et supprimer une note.

Nombre de fichiers. Partie client, 15 fichiers : `app/layout.tsx`, `app/page.tsx`, `app/globals.css`, `components/header.tsx`, `components/notes-board.tsx`, `components/note-card.tsx`, `components/note-editor.tsx`, `components/dictation.tsx`, 5 composants dans `components/ui/`, `hooks/use-whisper.ts`, `lib/i18n.ts`. Partie serveur, 10 fichiers : `app/actions.ts`, `lib/prefs.ts`, `db/index.ts`, `db/schema.ts`, `db/init.sql`, plus les 5 fichiers de configuration imposés par l'outillage Next.js et npm (`package.json`, `tsconfig.json`, `next.config.ts`, `postcss.config.mjs`, `.npmrc`). Le fichier `package-lock.json` est généré automatiquement par npm : il n'est pas écrit à la main mais il est fourni pour que l'installation donne exactement les mêmes versions.

2. Ce que fait le site et comment l'utiliser

Transcript.IO est un carnet de notes sur une seule page : un éditeur en haut, puis la liste des notes enregistrées en base, une par ligne, de la plus récente à la plus ancienne.

- **Créer une note** : saisir un titre (obligatoire), choisir une étiquette, écrire le contenu, puis « Enregistrer ».
- **Dicter** : « Dicter », autoriser le micro, parler, puis « Arrêter ». Le texte reconnu s'ajoute à la fin du contenu. La langue de dictée se choisit dans la liste à côté du bouton. Au premier usage, le modèle est téléchargé une fois puis mis en cache.
- **Modifier** : icône crayon sur une note, l'éditeur passe en « Modifier la note ». **Supprimer** : icône corbeille puis « Confirmer » (double clic volontaire pour éviter les suppressions accidentelles, sans fenêtre de confirmation du navigateur).
- **Changer de langue** : bouton « EN » / « FR » dans l'en-tête, ou dans le menu en version mobile. Toute l'interface, les dates et les noms d'étiquettes sont traduits.
- **Version mobile / bureau** : bouton « Version mobile » dans l'en-tête de la version bureau, « Version bureau » dans le menu de la version mobile. La version bureau occupe toute la largeur : titre et étiquette côte à côte, boutons compacts, réglages visibles dans l'en-tête et actions des notes qui ressortent au survol. La version mobile affiche une seule colonne, des boutons d'au moins 44 px de haut, un texte plus grand et un menu caché dans un tiroir de navigation (bouton ☰).
- **Thème clair / sombre** : bouton soleil / lune, ou dans le menu en version mobile.

3. Détails techniques

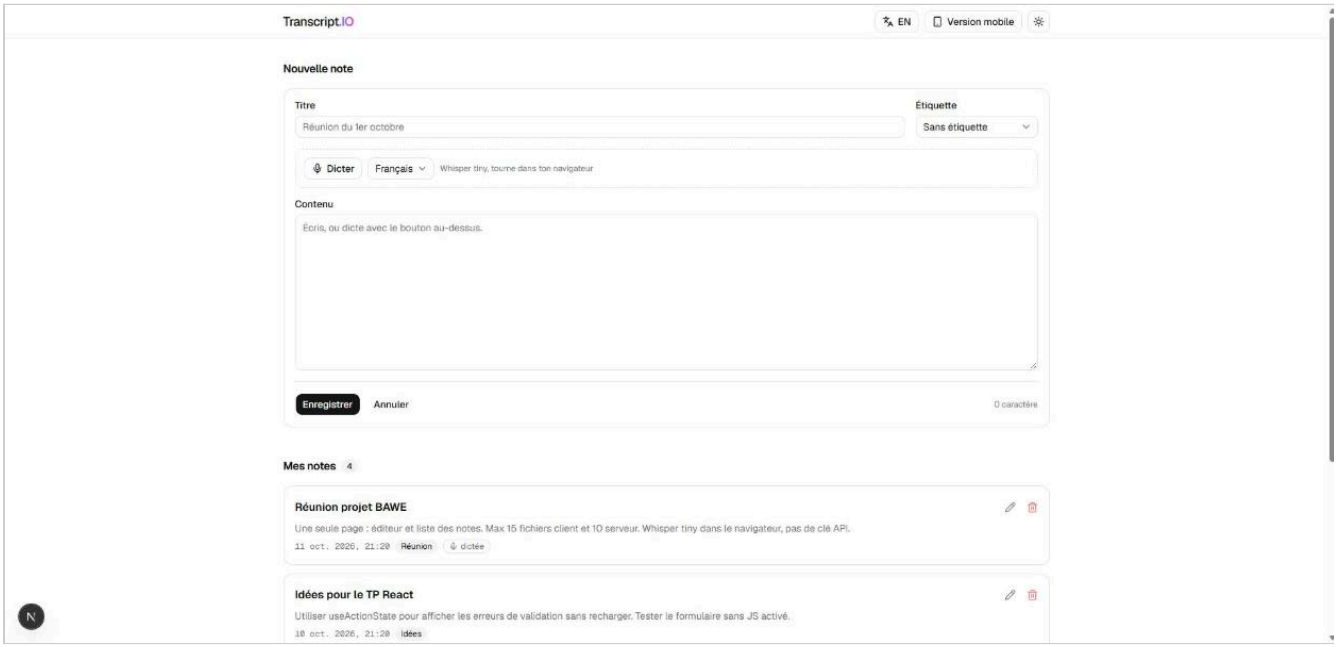
Bascule mobile / bureau sans dupliquer le code

Les classes « responsive » habituelles (`md:`) dépendent de la largeur de l'écran : elles ne permettent pas d'afficher la version mobile sur un ordinateur. J'ai donc défini deux variantes Tailwind dans `globals.css` : `desk:` et `mob:`, actives selon l'attribut `data-view` de la balise `<html>`. Le serveur fixe cet attribut d'après un cookie, ou d'après le navigateur (user-agent) si l'utilisateur n'a encore rien choisi. Le bouton appelle une Server Action qui écrit le cookie, et Next.js renvoie la page mise à jour sans rechargement complet. Les composants d'interface reçoivent une seule règle commune (`mob:min-h-11`) pour agrandir toutes les zones tactiles d'un coup. La langue fonctionne de la même façon (cookie `lang`), ce qui permet de traduire aussi les contenus générés côté serveur, comme les dates et les étiquettes lues en base.

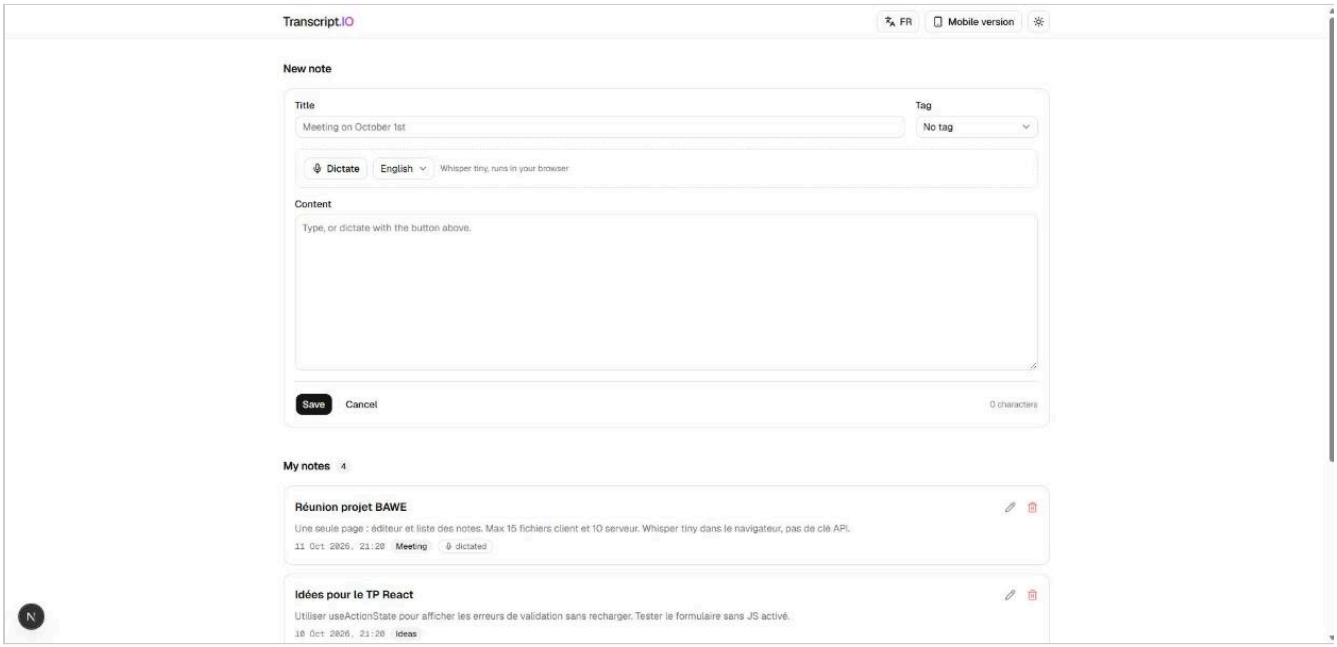
Transcription dans le navigateur

J'ai choisi d'exécuter Whisper localement plutôt que d'appeler une API payante : rien n'est envoyé à un serveur et aucune clé n'est nécessaire. La difficulté : la version multi-thread de WebAssembly exige que la page soit « cross-origin isolated ». J'ai donc ajouté les en-têtes `Cross-Origin-Opener-Policy` et `Cross-Origin-Embedder-Policy` dans `next.config.ts`. L'enregistrement du micro (`MediaRecorder`) est décodé par un `AudioContext` réglé à 16 kHz, le format attendu par Whisper, ce qui évite d'écrire un rééchantillonnage à la main.

Captures d'écran



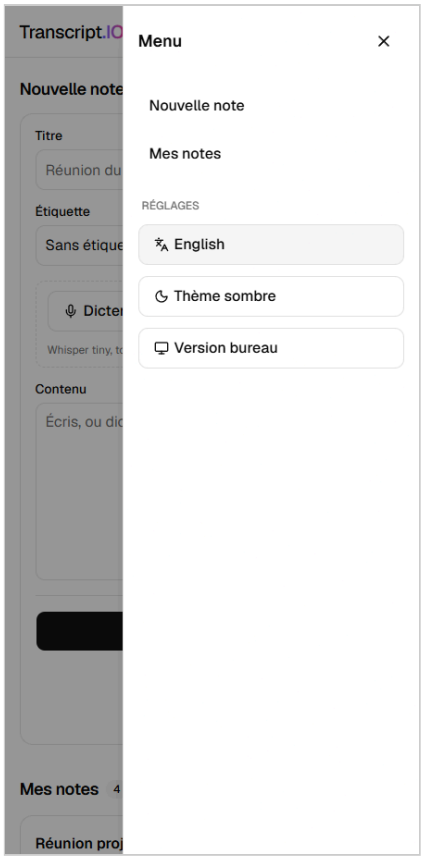
Version bureau, français



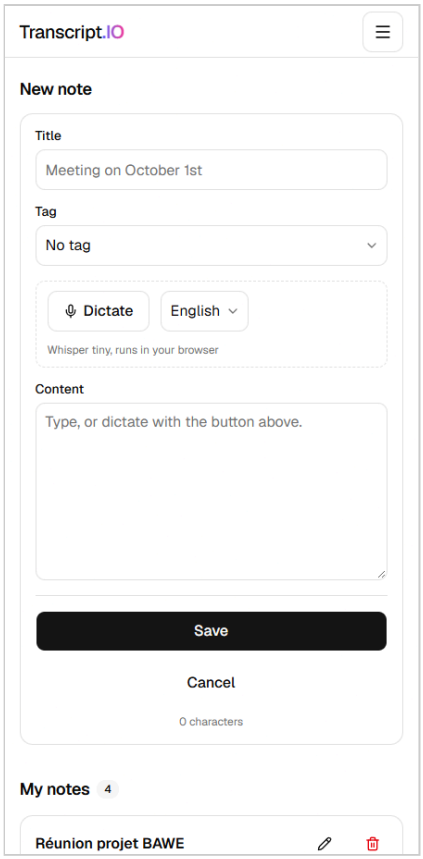
Version bureau, anglais



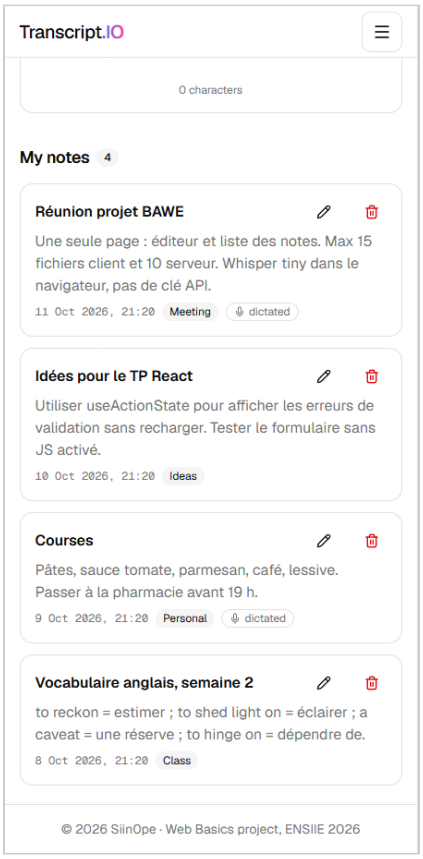
Version mobile, français



Mobile, tiroir de navigation ouvert



Version mobile, anglais



Mobile, liste des notes (anglais)